Unit 5: Numerical Methods

Prepared by Staples Education

Condensed Cheat Sheet

Unit 5 at a Glance: Core Sub-Topics

When an initial value problem $\frac{dy}{dt} = f(t, y),\ y(t_0) = y_0$ cannot be solved in closed form, we march the solution forward in small steps of size h. Every method in this unit is a recipe for turning the slope field f into the next point; they differ only in how many slopes they sample per step and how they weight them.

- **Euler's Method** — one tangent-line step $y_{n+1} = y_n + hf(t_n, y_n)$; first order, global error $O(h)$.
- **Improved and Modified Euler** — two-slope averaging and the midpoint step; both second order, $O(h^2)$.
- **Heun's Method** — the predictor–corrector reading of improved Euler as a trapezoidal average of endpoint slopes.
- **The RK4 Method** — four weighted stage slopes $\frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$; fourth order, $O(h^4)$.
- **The Runge–Kutta Family** — the general weighted-stage template that contains Euler, Heun, and RK4 as members.
- **Adaptive Step Size and Comparison** — estimating local error to resize h, and ranking the methods by order of accuracy.

Part I — Condensed Cheat Sheet

5.1 Euler's Method

Euler's method is the simplest numerical solver: from the current point it walks a single step along the tangent line whose slope is $f(t_n, y_n)$. It is the foundation every richer method refines.

The Euler Update

$$y_{n+1} = y_n + h\, f(t_n, y_n), \qquad t_{n+1} = t_n + h.$$

- Each step follows the **tangent line** with slope $f(t_n, y_n)$.
- **Local** truncation error is $O(h^2)$ (the dropped $\frac{h^2}{2}y''$ term); **global** error is $O(h)$, so Euler is a **first-order** method.
- Halving h roughly **halves** the global error.

Worked Example 5.1: One Euler step for $y' = y$

With $y(0) = 1$ and $h = 0.1$, the slope at the start is $f(0, 1) = 1$:

$$y_1 = 1 + 0.1 \cdot 1 = 1.1, \qquad t_1 = 0.1.$$

The exact value is $e^{0.1} \approx 1.10517$, so a single first-order step is already low by about 0.005.

5.2 Improved and Modified Euler Methods

Both upgrades sample a *second* slope to capture the curvature that plain Euler ignores, lifting the order from one to two. The improved method averages the slopes at the two endpoints; the modified (midpoint) method reads one slope at the interval's center.

Improved Euler (Endpoint Average) and Modified Euler (Midpoint)

Improved Euler:

$$k_1 = f(t_n, y_n), \qquad k_2 = f(t_n + h,\ y_n + hk_1)\,,$$

$$y_{n+1} = y_n + \frac{h}{2}\,(k_1 + k_2)\,.$$

Modified Euler (midpoint):

$$k_1 = f(t_n, y_n), \qquad k_2 = f\big(t_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_1\big)\,,$$

$$y_{n+1} = y_n + h\,k_2.$$

Both are **second order**, with global error $O(h^2)$: halving h cuts the error by about a factor of four.

Worked Example 5.2: Improved Euler for $y' = y$

With $y(0) = 1$, $h = 0.1$: the predictor slope is $k_1 = 1$ and the corrected slope is $k_2 = f(0.1,\ 1.1) = 1.1$. Averaging,

$$y_1 = 1 + \frac{0.1}{2}\,(1 + 1.1) = 1 + 0.05 \cdot 2.1 = 1.105.$$

This already matches $e^{0.1} \approx 1.10517$ to three decimals — far better than the plain Euler 1.1.

5.3 Heun's Method

Heun's method is improved Euler told as a *predictor–corrector*: take a provisional Euler step to predict the endpoint, then correct with the *average* of the slopes at the two ends. That average is exactly the trapezoidal rule applied to $\int f\,dt$.

Heun's Predictor–Corrector

1. **Predict** the endpoint with a plain Euler step:

$$\tilde{y}_{n+1} = y_n + h\,f(t_n, y_n).$$

2. **Correct** with the trapezoidal average of the two slopes:

$$y_{n+1} = y_n + \frac{h}{2}\Big[\,f(t_n, y_n) + f(t_{n+1}, \tilde{y}_{n+1})\Big].$$

Averaging the two endpoint slopes recovers the curvature term plain Euler drops, making Heun a **second-order** method — identical to improved Euler.

Common Pitfall: Unequal vs. Equal Slope Weights

The improved Euler / Heun corrector weights the two slopes *equally*, $\frac{h}{2}(k_1 + k_2)$. Do not borrow RK4's $\frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ weights for a two-slope method, and do not forget the predictor uses the *full* step h while the midpoint method uses a *half* step $\frac{h}{2}$.

5.4 The RK4 Method

The classical fourth-order Runge–Kutta method samples four slopes per step — at the start, twice at the midpoint, and once at the end — and blends them with Simpson-like weights $1, 2, 2, 1$. It is the workhorse of practical ODE solving.

The Classical RK4 Step

$$\begin{gathered}
k_1 = f(t_n,\, y_n), \\
k_2 = f\big(t_n + \tfrac{h}{2},\; y_n + \tfrac{h}{2}k_1\big)\,, \qquad k_3 = f\big(t_n + \tfrac{h}{2},\; y_n + \tfrac{h}{2}k_2\big)\,, \\
k_4 = f(t_n + h,\; y_n + hk_3)\,, \\
y_{n+1} = y_n + \frac{h}{6}\,(k_1 + 2k_2 + 2k_3 + k_4)\,.
\end{gathered}$$

RK4 is **fourth order**, with global error $O(h^4)$: halving h shrinks the error by about a factor of **16**.

Worked Example 5.4: One RK4 step for $y' = y$

With $y(0) = 1$, $h = 0.1$:

$$k_1 = 1, \quad k_2 = 1.05, \quad k_3 = 1.0525, \quad k_4 = 1.10525.$$

$$y_1 = 1 + \frac{0.1}{6}\big(1 + 2(1.05) + 2(1.0525) + 1.10525\big) = 1.105171.$$

Against $e^{0.1} = 1.1051709\ldots$ the error is below 10^{-6} in a single step.

5.5 The Runge–Kutta Family

Euler, the midpoint method, Heun, and RK4 are all instances of one template: take a weighted average of slopes sampled at chosen points across the step. The *number of stages* and the *weights* fix the order of accuracy.

The General Explicit Runge–Kutta Step

For s stages with weights b_i, nodes c_i, and coupling coefficients a_{ij}:

$$k_i = f\left(t_n + c_i h,\ y_n + h \sum_{j<i} a_{ij}\, k_j\right), \qquad y_{n+1} = y_n + h \sum_{i=1}^{s} b_i\, k_i.$$

- Consistency requires $\sum_i b_i = 1$ (the weights sum to one).
- Euler is the 1-stage member; improved Euler, the midpoint method, and Heun are 2-stage second-order members; RK4 is the classic 4-stage fourth-order member.

5.6 Adaptive Step Size and Method Comparison

Rather than fix h in advance, adaptive solvers *estimate* the local error each step and resize h to hold it under a tolerance — small steps through sharp turns, large steps across calm stretches.

Estimating Local Error to Resize h

1. Advance the step two ways of *different order* (e.g. an embedded RK4/RK5 pair, or one full step vs. two half steps).
2. The difference of the two estimates approximates the **local error**.
3. If the error exceeds the tolerance, **shrink** h and retry; if it is comfortably under, **grow** h for the next step.

Common Pitfall: Confusing Order with Step Count

A method's *order* p is how fast error falls as $h \to 0$ (global error $O(h^p)$), not how many steps it takes. RK4 with a large h can beat Euler with a tiny h at equal cost. And smaller h is not free: too small, and accumulated roundoff begins to swamp the truncation gain.

Unit 5 Key Takeaway

Every solver here marches $y_{n+1} = y_n + h\,$(weighted slope); they differ only in the slope. Euler samples one slope (first order, $O(h)$); improved/modified Euler and Heun average two (second order, $O(h^2)$); RK4 blends four with weights $1, 2, 2, 1$ (fourth order, $O(h^4)$). All sit inside the Runge–Kutta family, and adaptive stepping tunes h from an on-the-fly local-error estimate.